

面向 LLM / Agent Systems 科研的深度学习零基础课

第 07 讲

神经元究竟是什么：Weight、Bias 与加权求和

本讲目标：把“一个人工神经元”从神秘名词还原成透明的数字计算。真正理解输入 x 、权重 w 、偏置 b 、加权贡献、pre-activation z ，并理解多个神经元为什么自然变成矩阵乘法。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

1 为什么这一讲要把“神经元”重新拆开	3
1.1 本讲最低验收标准	3
2 【原理】一个神经元到底接收什么？	4
3 【通俗解释】Weight 可以先理解成“调节旋钮”	4
4 【具体数字例子】完整手算一个神经元	5
4.1 第一步：逐项相乘	5
4.2 第二步：求和	5
4.3 第三步：加 Bias	5
5 【必要公式】从求和形式到向量形式	6
6 为什么需要 Bias？	6
6.1 一个一维例子	7
7 Weight 改一点，z 会怎样变？	7
8 一个神经元能做什么，不能做什么？	8
9 【图解】多个神经元为什么自然变成一层？	8
10 Batch 中发生了什么？	9
11 参数数量怎样算？	9
12 【代码实验】先用 NumPy，不让框架藏住细节	10
13 【实验观察】改变输入、Weight、Bias 分别意味着什么	11
13.1 干预 1：只改变一个输入	11
13.2 干预 2：比较正负 Weight	11
13.3 干预 3：只改 Bias	11
14 【常见误区】几个必须现在纠正的理解	11
14.1 误区 1：神经元就是生物神经元的精确复制	11
14.2 误区 2：Weight 越大，该特征越重要	11
14.3 误区 3：Bias 就是模型误差	11

14.4 误区 4：一个神经元已经可以表达任意复杂规律	12
14.5 误区 5：训练时人手工指定每个 Weight	12
15 【与 LLM / Transformer / Agent Systems 的关系】	12
15.1 Transformer 里的 Linear 可以看成大量神经元并排	12
15.2 FFN/MLP 更直接	12
15.3 Agent Systems 为什么也需要懂这一层?	12
16 【科研视角】参数是“可学习数字”，不是魔法知识块	13
17 本讲完整心智链	13
18 本讲检查点	13
19 下一讲预告	14

1 为什么这一讲要把“神经元”重新拆开

你以前在 LLM 课程里已经见过“神经网络、参数、Weight、Bias”等词，但如果脑中只留下“神经网络就是很多层”，后面学习 MLP、Activation、Backpropagation 时仍然会缺一块底座。

这一讲只做一件事：**把一个神经元的计算彻底透明化。**

本讲核心句

一个最基本的人工神经元，在进入激活函数之前，先做：

$$z = w_1x_1 + w_2x_2 + \cdots + w_Dx_D + b$$

也可以写成：

$$z = x \cdot w + b$$

它本质上就是：**每个输入乘一个权重 -> 全部相加 -> 再加一个偏置。**

与 LLM / Transformer / Agent Systems 的连接

你以前在 LLM 里见过参数量巨大、Transformer 中大量 Linear 层。那些“几亿、几十亿参数”并不是神秘物质，本质上就是大量可训练的 Weight 与 Bias（再加上 Embedding 等参数）组成。今天我们先看最小单位：一个神经元的一组 Weight 和一个 Bias。

1.1 本讲最低验收标准

学完后你应该能独立做到：

1. 解释一个神经元的输入、权重、Bias 和输出 z ；
2. 手算 $z = x \cdot w + b$ ；
3. 解释每个 x_iw_i 的数值意义；
4. 解释 Weight 正负号与绝对值怎样影响当前线性响应；
5. 解释 Bias 为什么不是“多余的一项”；
6. 从一个神经元推广到多个神经元，写出 $z = xW + b$ ；
7. 区分“参数的数值作用”和“现实世界的因果重要性”；
8. 看懂一个简单 `nn.Linear` 在计算什么；
9. 说明为什么目前还没有得到真正的非线性深层网络。

2 【原理】一个神经元到底接收什么？

假设一个样本用三个数字描述：

$$x = [x_1, x_2, x_3]$$

例如某个极简“科研论文是否容易被接受”的玩具模型，把三个输入理解成：

- x_1 ：实验完整度；
- x_2 ：复现难度；
- x_3 ：问题的重要程度。

这里的例子只是为了理解数字，不代表真实科研评价模型。

一个神经元不会直接说“第一个特征重要、第二个不重要”。它为每个输入准备一个数字：

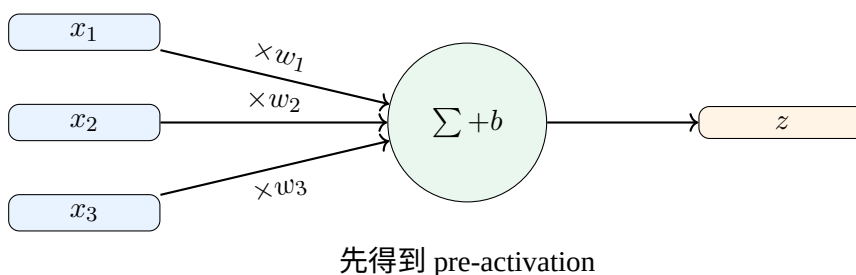
$$w = [w_1, w_2, w_3]$$

这些就是 **Weight（权重）**。

然后再准备一个额外的标量：

$$b$$

这就是 **Bias（偏置）**。



3 【通俗解释】Weight 可以先理解成“调节旋钮”

为了建立直觉，可以暂时把每个 Weight 想成一个调节旋钮。

输入 x_i 来了以后，神经元不会原样使用，而是先乘：

$$x_i w_i$$

- $w_i > 0$ ：输入增加时，会推动当前 z 往更大的方向变化；
- $w_i < 0$ ：输入增加时，会推动当前 z 往更小的方向变化；
- $|w_i|$ 较大：同样的输入变化，会引起更大的线性响应；
- $w_i = 0$ ：在这一层当前计算中，该输入对这个神经元的直接贡献为 0。

先不要把 Weight 当成 “真实世界重要性”

“权重大”只是在特定模型、特定输入尺度、特定层和训练结果下的数值性质。它**不自动等于现实因果重要性**，也不保证可以直接解释成“这个因素最重要”。后面做科研时，模型解释需要更严格的方法。

4 【具体数字例子】完整手算一个神经元

设输入：

$$x = [2, -1, 3]$$

权重：

$$w = [0.5, -2, 1]$$

Bias：

$$b = 0.2$$

计算：

$$z = x \cdot w + b$$

4.1 第一步：逐项相乘

$$2 \times 0.5 = 1$$

$$(-1) \times (-2) = 2$$

$$3 \times 1 = 3$$

因此三个直接贡献为：

$$[1, 2, 3]$$

4.2 第二步：求和

$$1 + 2 + 3 = 6$$

4.3 第三步：加 Bias

$$z = 6 + 0.2 = 6.2$$

这里每个数字分别是什么？

- x_i : 这个样本的输入特征；
- w_i : 模型内部可学习参数；
- $x_i w_i$: 该输入在当前神经元线性计算中的直接数值贡献；
- b : 与输入无关的额外平移项；
- z : 激活函数之前的结果，因此常叫 *pre-activation*。

5 【必要公式】从求和形式到向量形式

单神经元最常见公式：

$$z = \sum_{i=1}^D x_i w_i + b$$

等价的向量写法：

$$z = x \cdot w + b$$

如果使用列向量约定，也常写：

$$z = w^T x + b$$

公式逐项拆解

- D : 输入特征数量；
- x_i : 第 i 个输入；
- w_i : 与第 i 个输入对应的权重；
- $\sum$: 把所有加权贡献加起来；
- b : Bias；
- z : 一个标量，因为这里讨论的是一个神经元。

6 为什么需要 Bias?

先看没有 Bias：

$$z = x \cdot w$$

如果所有输入都是 0：

$$x = [0, 0, \dots, 0]$$

那么一定得到：

$$z = 0$$

加入 Bias:

$$z = x \cdot w + b$$

即使 $x = 0$ ，仍然可以:

$$z = b$$

Bias 的直觉

Weight 决定“输入变化如何推动输出”，Bias 决定“整体基线放在哪里”。从几何角度看，Bias 允许模型把线性边界从必须经过原点的位置平移出去。

6.1 一个一维例子

如果:

$$z = 2x$$

当 $x = 0$ 时只能有 $z = 0$ 。

如果:

$$z = 2x + 3$$

整条直线向上平移 3。

Bias 不是“误差”

中文“偏置”容易让初学者误以为 Bias 是模型犯的错误。不是。Bias 是模型主动学习的参数，是模型表达能力的一部分。

7 Weight 改一点，z 会怎样变？

仍然考虑:

$$z = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

如果只让 x_1 增加 Δx_1 ，其余不变，那么:

$$\Delta z = \Delta x_1 \cdot w_1$$

例如:

$$w_1 = 0.5$$

让 x_1 从 2 增加到 4，变化量是 2，因此:

$$\Delta z = 2 \times 0.5 = 1$$

这正是本讲实验会验证的内容。

这和以后 Gradient 有什么伏笔？

当前式子里， z 对 x_i 的局部变化率就是 w_i 。以后学习导数与 Gradient 时，会把“一个变量改变一点，另一个量怎样变化”正式数学化。现在只需要建立直觉。

8 一个神经元能做什么，不能做什么？

一个神经元的 pre-activation：

$$z = x \cdot w + b$$

本质仍然是**线性/仿射映射**。

它可以把多个输入压缩成一个分数，但能力有限。如果我们只有这样的线性计算，不管堆多少层，仍然存在一个重要限制：**缺少非线性表达能力**。

这个问题将在第 8、9 讲正式展开。

不要提前把 z 当成最终“神经元输出”

很多教材口语上把 z 也称输出，但严格来说，在含激活函数的神经元中：

$$z = x \cdot w + b$$

只是 pre-activation；真正传给下一层的通常是：

$$a = f(z)$$

其中 f 是 Activation Function。我们会在后续专门学习。

9 【图解】多个神经元为什么自然变成一层？

如果只有一个神经元：

$$x : (D,), \quad w : (D,), \quad z : (1)$$

如果并排放 2 个神经元，每个都有自己的权重：

$$w^{(1)}, \quad w^{(2)}$$

把它们并排放成矩阵：

$$W = \begin{bmatrix} | & | \\ w^{(1)} & w^{(2)} \\ | & | \end{bmatrix}$$

那么：

$$W : (D, 2)$$

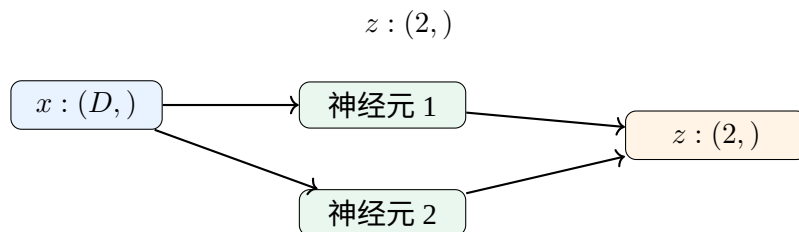
Bias 也变成：

$$b : (2,)$$

于是两个神经元可以一次计算：

$$z = xW + b$$

得到：



从“神经元”到“矩阵乘法”的关键跃迁

一个神经元是一个 $x \cdot w + b$ 。很多神经元并排，就是把很多 w 组织成矩阵 W ，一次执行 $xW + b$ 。这就是上一讲 Linear Layer 的“微观解释”。

10 Batch 中发生了什么？

如果不是一个样本，而是：

$$X : (B, D)$$

有 O 个神经元：

$$W : (D, O), \quad b : (O,)$$

那么：

$$Z = XW + b : (B, O)$$

每一行是一个样本，每一列对应一个神经元。

具体例子

若：

$$X : (32, 768), \quad W : (768, 3072)$$

则：

$$Z : (32, 3072)$$

可以理解成：32 个样本同时经过 3072 个神经元，每个神经元都接收 768 个输入值。

11 参数数量怎样算？

一个 $D \rightarrow O$ 的线性层中：

- Weight 数量： $D \times O$ ；

- Bias 数量： O ；
- 总参数量：

$$D \times O + O$$

如果只有一个神经元，即 $O = 1$ ：

$$D \times 1 + 1 = D + 1$$

连接你以前学过的 LLM 参数

Transformer FFN、Attention 投影等模块中，大量参数都来自这种 Linear 权重矩阵。例如一个 $4096 \rightarrow 11008$ 的 Linear，仅 Weight 就有：

$$4096 \times 11008$$

个可学习数字。大模型参数量巨大的一个根本原因，就是隐藏维度很大、层数很多、矩阵很多。

12 【代码实验】先用 NumPy，不让框架藏住细节

本讲主实验文件：

```
lesson07_neuron_experiment.py
```

最核心代码只有：

```
import numpy as np

x = np.array([2.0, -1.0, 3.0])
w = np.array([0.5, -2.0, 1.0])
b = 0.2

contributions = x * w
z = contributions.sum() + b

print(contributions)
print(z)
```

输出：

```
[1.  2.  3.]
6.2
```

也可以直接写：

```
z = x @ w + b
```

这两种写法数值完全等价。前者更透明，后者更接近真实深度学习代码。

13 【实验观察】改变输入、Weight、Bias 分别意味着什么

本讲实验会做三类干预。

13.1 干预 1：只改变一个输入

保持 Weight 不动，改变 x_1 。目的是观察：

$$\Delta z = \Delta x_1 w_1$$

13.2 干预 2：比较正负 Weight

让每个输入分别增加 1：

- $w_1 = 0.5$ ， z 增加 0.5；
- $w_2 = -2$ ， z 减少 2；
- $w_3 = 1$ ， z 增加 1。

13.3 干预 3：只改 Bias

让 b 在 $-2, 0, 0.2, 3$ 之间变化，观察 z 整体平移。

科研视角

这其实已经是一个最小“控制变量实验”：只改一个因素，其他保持不变，观察输出变化。后续做深度学习实验时，很多 Ablation 的基本逻辑也是如此，只是系统复杂得多。

14 【常见误区】几个必须现在纠正的理解

14.1 误区 1：神经元就是生物神经元的精确复制

不是。人工神经元只是受生物神经系统启发的数学计算单元。现代深度学习中的“神经元”主要是数学和工程抽象。

14.2 误区 2：Weight 越大，该特征越重要

不一定。输入尺度、Normalization、后续层、非线性、特征相关性都会影响解释。

14.3 误区 3：Bias 就是模型误差

不是。Bias 是可学习参数。

14.4 误区 4：一个神经元已经可以表达任意复杂规律

不能。一个神经元目前只是一个仿射计算，再配合简单激活。复杂能力来自大量单元、层级组合、非线性和训练。

14.5 误区 5：训练时人手工指定每个 Weight

真实深度学习中，Weight 通常由训练过程自动调整。后面学习 Loss、Gradient、Backpropagation 时会回答“它们如何自己变”。

15 【与 LLM / Transformer / Agent Systems 的关系】

15.1 Transformer 里的 Linear 可以看成大量神经元并排

例如：

$$Q = XW_Q$$

对每个 Token 来说， W_Q 中的不同输出列可以看作不同线性单元，它们共同把原来的 hidden representation 映射成 Query representation。

15.2 FFN/MLP 更直接

Transformer 中的 Feed-Forward Network 通常有类似：

$$h \rightarrow hW_1 + b_1 \rightarrow \text{Activation} \rightarrow \dots$$

这里第一步就是本讲内容的批量版本。

15.3 Agent Systems 为什么也需要懂这一层？

Agent 系统研究不等于训练基础模型，但你会频繁读到：

- base model parameters;
- frozen / trainable parameters;
- LoRA adapter;
- MLP / projection;
- hidden dimension;
- embedding projection。

理解 Weight、Bias、Linear 的底层含义，才能判断这些模型改动究竟动了什么。

16 【科研视角】参数是“可学习数字”，不是魔法知识块

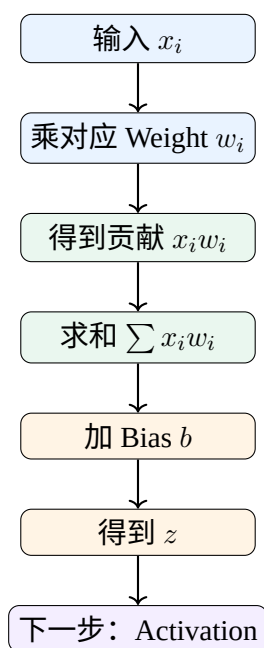
一个很重要的研究直觉是：模型“知道某件事”并不意味着存在一个单独参数专门存“这件事”。知识通常分布在大量参数与激活模式中。

因此以后看到：

- “修改某个参数”；
- “冻结某些层”；
- “只训练 Adapter”；
- “剪枝某些权重”；

你要先问：这会改变哪些数值计算路径？而不是把参数想成数据库里一条条独立知识记录。

17 本讲完整心智链



18 本讲检查点

如果下面 6 个问题都能口头回答，就可以继续

1. $x = [2, -1, 3]$ 、 $w = [0.5, -2, 1]$ 、 $b = 0.2$ 时，为什么 $z = 6.2$ ？
2. 为什么 x 与 w 必须有对应的相同长度？
3. Weight 为负数意味着什么？
4. Bias 的作用为什么不能简单被 Weight 取代？

5. 为什么 100 个神经元可以组织成一次矩阵乘法？
6. 为什么 Weight 大不能直接推出“现实因果重要性大”？

19 下一讲预告

现在我们已经有了：

$$z = xW + b$$

但一个关键问题马上出现：

如果所有层都只是 Linear，堆很多层真的会更强吗？

第 08 讲将回答这个问题：为什么仅有 Linear 不够，以及为什么神经网络必须引入非线性。